

Helm & Templating

Package Management for Kubernetes — Module 12 of 13

Enterprise EKS Platform



Agenda

Core Concepts

- Why Helm — the problem it solves
- Charts, releases, and repositories
- Chart structure and values
- Go template language

Operations & Advanced

- Install, upgrade, rollback lifecycle
- Hooks, dependencies, and testing
- Helm vs Kustomize
- Best practices & Lab 12

Why Helm?

The problem with raw YAML at scale

The YAML Sprawl Problem

A typical microservice requires 5–10 YAML manifests — Deployment, Service, ConfigMap, Secret, Ingress, HPA, PDB, ServiceAccount, NetworkPolicy...

- **Duplication** — same patterns repeated across dozens of services
- **Environment drift** — dev, staging, and prod configs diverge over time
- **No versioning** — hard to track what's deployed or roll back atomically
- **No dependency management** — ordering and relationships are manual
- **Parameterization** — sed and envsubst don't scale

Helm Core Concepts

Chart

A package of pre-configured Kubernetes resources — templates + default values + metadata

Release

A running instance of a chart in a cluster — each install creates a new release with its own history

Repository

A server hosting packaged charts — similar to Docker Hub for images or npm for packages

Key relationship: A *chart* is installed into a cluster as a *release*. The same chart can be installed multiple times, each producing a distinct release.



Helm Architecture

Chart structure, values, and the rendering pipeline

Values and Override Hierarchy

Helm merges values from multiple sources in precedence order:

Override Precedence (lowest to highest)

1. **Parent chart's values.yaml** — defaults from dependency charts
2. **Chart's values.yaml** — the chart's own defaults
3. **-f custom-values.yaml** — user-supplied values files
4. **--set key=value** — command-line overrides (highest priority)



Templating

Go templates, functions, and control flow

Built-in Objects

.Values

Contents of `values.yaml` merged with overrides

.Chart

Contents of `Chart.yaml` — `.Name`, `.Version`, `.AppVersion`

.Release

`.Name`, `.Namespace`, `.IsInstall`, `.IsUpgrade`, `.Revision`

.Capabilities

`.KubeVersion`, `.APIVersions` — detect cluster features at render time

Tip: Use `.Capabilities.APIVersions.Has` to conditionally include resources based on what the target cluster supports.

Named Templates & `_helpers.tpl`

What they are

Reusable snippets defined in `_helpers.tpl` (underscore prefix means Helm won't render it as a manifest) — common patterns like label blocks defined once and shared across resources.

Use `include over template` — `include` captures output as a string so you can pipe it through functions like `nindent`.

Helm Operations

Install, upgrade, rollback, and release management

Release Lifecycle

Core commands

- **install** — deploy a chart as a new release
- **upgrade** — apply changes, creating a new revision
- **rollback** — revert to a previous revision's config and templates
- **uninstall** — remove a release and its resources

Best practice: Always use `helm upgrade --install` in CI/CD pipelines for idempotent deployments.

Debugging & Validation

helm template

Renders manifests locally without cluster access — supports overrides and single-template selection

helm lint

Checks chart structure and best practices — strict mode treats warnings as errors

Use in CI: `helm lint + helm template` form the first line of defense in chart validation.

Release Management & History

Release state is stored as Secrets in the release namespace. Each revision creates a new Secret named `sh.helm.release.v1.<name>.v<revision>`.

Advanced Helm

Hooks, dependencies, library charts, and testing

Helm Hooks

Hooks are regular Kubernetes resources with special annotations that run at specific points in the release lifecycle. `hook-weight` controls order; `hook-delete-policy` handles cleanup.

Common use cases: Database migrations before upgrade, cache warming after install, cleanup jobs before delete.

Helm Hook Types

Hook	When It Runs
pre-install	Before resources are created
post-install	After all resources are created
pre-upgrade / post-upgrade	Before/after upgrade
pre-rollback / post-rollback	Before/after rollback
pre-delete / post-delete	Before/after uninstall

hook-weight controls execution order when multiple hooks exist. **hook-delete-policy** controls whether hook resources are cleaned up automatically.

Chart Dependencies

Dependencies are declared in `Chart.yaml` with a name, version, and repository, then downloaded into `charts/` as tarballs.

condition and **tags** let users enable/disable dependencies. Use **alias** to install the same chart multiple times with different configs.

Subchart values are scoped under the dependency name (or alias) in the parent's `values.yaml`. `Chart.lock` pins exact versions.

Library Charts

What are Library Charts?

Charts with type: `library` that provide **only named templates** — no rendered manifests. Declared as a dependency and consumed via `include` in consumer charts.

Use case: Enforce org-wide standards for labels, security contexts, and resource structures across services.

Helm Testing

How it works

Tests are Pods annotated with `helm.sh/hook: test`, defined under `templates/tests/`. Run them with `helm test <release>` to verify connectivity and functionality after install or upgrade.

CI/CD integration: Include `helm test` in your pipeline after every deployment to catch issues before they reach users.

OCI Registry Support

Since Helm 3.8+, charts can be stored in OCI-compliant container registries alongside your container images — package and push with `helm push`, then install directly from an `oci://` reference.

Advantages: Unified artifact storage, immutable versions, standard auth, no separate chart server needed.

Helm vs Kustomize

Feature	Helm	Kustomize
Approach	Templating (generate YAML)	Patching (overlay on base YAML)
Parameterization	Go templates + values	Strategic merge patches
Packaging	Charts with versioning	No packaging concept
Release tracking	Built-in (Secrets)	None (relies on GitOps)
Rollback	Built-in (<code>helm rollback</code>)	Via Git revert
Learning curve	Steeper (Go templates)	Moderate (YAML patches)
Built into kubectl	No (separate binary)	Yes (<code>kubectl -k</code>)

They complement each other. Many teams use Helm for packaging third-party charts and Kustomize for environment-specific overlays on top of `helm template` output.

Best Practices

Chart development guidelines and recommended patterns

Chart Development Guidelines

Naming & Structure

- Follow [Helm best practices guide](#)
- Use `helm create` to scaffold new charts
- Prefix template names with chart name
- Keep templates focused — one resource per file

Values & Defaults

- Provide sensible defaults for all values
- Use flat keys where possible (avoid deep nesting)
- Document every value with comments
- Never put secrets in `values.yaml`

Version your charts independently from your application. Bump the chart version for template changes, and `appVersion` for the application itself.

Values Schema: Validation Benefits

- **Type checking** — catch string vs number errors
- **Required fields** — fail fast on missing values
- **Range validation** — enforce min/max constraints
- **Enum validation** — restrict to allowed values
- **Pattern matching** — validate format (e.g., semver)

Validated automatically during `helm install`, `helm upgrade`, `helm lint`, and `helm template`.



Module Summary

- Helm is a package manager for Kubernetes using charts, templates, and values
- Go templates with values.yaml enable reusable, parameterized deployments
- Named templates in _helpers.tpl promote DRY patterns across resources
- Helm tracks release history enabling rollback to any previous revision
- OCI registry support stores charts alongside container images
- Helm and Kustomize complement each other for packaging and environment overlays



Lab 12: Helm and Templating

Estimated Time: ~45 minutes

- Install and explore a chart from the podinfo repository
- Upgrade a release and rollback to a previous revision
- Create a custom Helm chart using `helm create`
- Add custom templates with Go templating and values
- Validate with `helm lint`, `helm template`, and dry-run
- Package and install the custom chart

Key Takeaways

Remember These Points

1. Use `helm create` to scaffold charts and follow the official best practices guide
2. Version charts independently from applications and never put secrets in `values.yaml`
3. Add a `values.schema.json` for validation to catch configuration errors before deployment

? Questions & Discussion

Module 12: Helm & Templating

Up Next: Module 13 — CI/CD Integration & GitOps